CS 3960 (3)

Vibe Coding

Spring 2026, *content* Pavel Panchekha and John Regehr, *illustrations* Gemini 3 Nano Banana Pro

- How's the latency assignment coming along?
- Reading assignment due Friday
  - When AI Writes the World's Software, Who Verifies It?
  - Answer the questions

Today:

Intro to formal verification of software

Although we often don't think about it this way, **computer programs are math**

And, using math, we can prove things about computer programs

AI is making formal verification into a key concern rather than a niche topic for academics and safety-critical systems

We might want to prove that:

- Compressing and decompressing a file gives back the original bytes
- Operating system user A can never read memory or files belonging to user B
- Our application never crashes
- Our application never executes code besides what was shipped

Blueprint for all formal verification efforts:

1. Write a specification for what the program should or shouldn’t do
2. Prove that the program meets the specification

Some specifications are pretty simple: “Don’t crash”

Other specifications are very, very difficult:

“Be a correct C++20 compiler”

“Be a correct air traffic control system”

We’re not going to look at specifications in very much detail

But… they’re just math

What does it mean to prove something mathematically?

We can understand programs using math

However, the behavior of programs often diverges from the math we all learned in school

In math, it is often the case that:

$(x+y)+z = x+(y+z)$

In IEEE floating point math:

$(x+y)+z \mathrel{!}= x+(y+z)$

In IEEE float point math we have both +0 and -0

What does x+1 mean in math, where x is a natural number?

What are some of the things that x+1 might mean, where x is a computer integer?

In C and C++:

```
(x+1)  >  x
```

Is always true when x is a signed integer (but in a tricky way).

But might be false when x is an unsigned integer.

The point:

To prove things about computer programs, we have to have an extremely detailed understanding of what programming languages actually mean.

Seems like kind of bad news.

But there is good news:

We can offload understanding programming languages to tools.

Now the workflow is:

1. Write a specification in a specification language
2. Ask a tool to verify that our implementation meets this spec

We'll talk a bit more about tools later.

For now, let's reason about programs using our brains.

Doing lots of little proofs in your head is one of the best ways to write better software.

What do these little proofs look like?

At some point in our program, we'll want to prove something like:

- This pointer is not null
- This array index is in bounds
- This file is open for reading
- This thread has exclusive access to a data structure

How do you do these proofs?

You work backwards: Look at the code leading up to the point in the program you care about, and you reason about its effects.

You have to look at all possible ways to get there.

One of the core ways we reason about code formally is using “Hoare triples”:

{ P } — a precondition

S — a statement

{ Q } — a postcondition

Then we chain these together across statements

Let's prove an implementation of "popcount" correct. This function returns the number of set bits in its argument.

```
popcount(0) = ?
popcount(7) = ?
popcount(8) = ?
```

```
int popcount(uint64_t n) {
  int count = 0;
  while (n) {
    n = n & (n - 1);
    ++count;
  }
  return count;
}
```

What's the specification?

Let's just assume we have a reference, popcount-ref().

So, we're trying to prove that

popcount-ref(x) = `popcount(x)`

... for all 2^{64} possible values of x

Precondition?

Postcondition?

```
int popcount(uint64_t n) {
  int count = 0;
  while (n) {
    n = n & (n - 1);
    ++count;
  }
  return count;
}
```

Precondition?

Postcondition?

```
int popcount(uint64_t n) {
  int count = 0;
  while (n) {
    n = n & (n - 1);
    ++count;
  }
  return count;
}
```

Precondition?

Postcondition?

```
int popcount(uint64_t n) {
  int count = 0;
  while (n) {
    n = n & (n - 1);
    ++count;
  }
  return count;
}
```

Why does `n & (n - 1)` clear the lowest set bit?

`????1000` <— n

`????0111` <— n-1

`????0000` <— n & (n-1)

Higher bits are unaffected

Precondition?

Postcondition?

```
int popcount(uint64_t n) {
  int count = 0;
  while (n) {
    n = n & (n - 1);
    ++count;
  }
  return count;
}
```

Ok we're kind of stuck...

From what we've seen to far, Hoare logic won't get us any further

We'll need an insight about the loop's purpose to progress